DATA ENGINEERING

Time-Series Databases Visual Guide

Metrics, sensors, and ticks.
Built for data that never stops arriving.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Shape of Time-Series Data

Timestamp + tags + fields. Append-only, almost never updated.

```
cpu,host=web-01,region=us-east usage=42.1 1728400000
cpu,host=web-02,region=us-east usage=38.7 1728400000
cpu,host=web-01,region=us-east usage=44.0 1728400060
```

Timestamp
the primary key

Tags
indexed dimensions

Fields
measured values

Tags (host, region) are indexed for filtering; fields (usage) are the values you aggregate. Cardinality of tags is the scaling cliff — watch it.

STEP 2/6

Why Not Just Use Postgres?

Time-series workloads break general-purpose databases at scale.

- Write rate: millions of points/sec — B-tree indexes can't keep up
- Queries: time-range scans + downsampling, not point lookups
- Data value decays: last week's per-second data is worth less than last year's per-hour
- Compression: sorted-by-time numeric series compress 10-20x with delta encoding

TSDBs trade general query power for insane ingest and time-range performance: LSM trees, columnar chunks, time-partitioned storage.

STEP 3/6

InfluxDB vs TimescaleDB

Purpose-built engine vs Postgres superpowers.

InfluxDB

purpose-built TSDB

- Flux/SQL query language
- Built-in retention + downsampling
- Best for: metrics, IoT at scale

TimescaleDB

Postgres extension

- Full SQL + joins to relational data
- Hypertables auto-partition by time
- Best for: teams that live in Postgres

Choose InfluxDB for pure time-series scale; TimescaleDB when time-series is one workload among many and you want real SQL joins.

STEP 4/6

Downsampling: Rollups Over Time

Keep full resolution briefly, aggregates forever.

Continuous aggregates (Timescale) or tasks (InfluxDB) compute rollups on ingest. Dashboards query the rollup — raw data is for forensics.

STEP 5/6

Retention Policies

Data expires by policy, not by accident.

```
CREATE RETENTION POLICY '30d' ON metrics
  DURATION 30d REPLICATION 1;
```

- Tier by age: hot (SSD, full resolution) → warm → cold (object storage)
- Retention runs per measurement — different series, different lifespans
- Legal hold overrides: compliance data ignores default expiry
- Dropping whole time-partitions is O(1) — never DELETE old rows one by one

Retention is a cost control: per-second data for 30 days, then rollups. Without policies, cardinality and storage grow until the bill does.

STEP 6/6

Compression: Why TSDBs Are Tiny

Time-ordered numerics compress absurdly well.

Delta-of-delta

timestamps: store diffs of diffs

Gorilla / XOR

floats: store changed bits only

Regular 60s timestamps: deltas are constant → near-zero bits.
Slowly-changing floats: XOR with previous value leaves
mostly zeros. Result: 10-20x smaller than raw.

Compression is why TSDBs can hold years of metrics cheaply — but it depends on time-ordered writes. Out-of-order ingestion costs you.

STEP ✓I✓

Key Takeaways

Time-Series Databases — Remember

- Time-series data = timestamp + tags (indexed) + fields (measured); append-only and high-volume.
- TSDBs beat general databases on ingest rate, time-range scans, and automatic lifecycle management.
- InfluxDB for pure TS scale; TimescaleDB when you need real SQL and joins to relational data.
- Downsample aggressively: raw for days, minute-aggregates for months, hourly for years.
- Retention policies + time-ordered compression (delta, Gorilla) keep years of data affordable.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev